

SRE Bot - Architecture Analysis & Design

Executive Summary

This document outlines the architecture and implementation plan for an enterprise-ready AI Agentic SRE Bot that supports multiple LLM providers (OpenAI, Anthropic, Ollama) with strict data residency and security constraints.

Key Requirements

Functional Requirements

1. **Multi-LLM Support:** OpenAI, Anthropic (MVP), Ollama (local hosting)
2. **Agentic Capabilities:** Tool use, reasoning, planning, execution
3. **SRE Operations:** Network troubleshooting, observability, migration assessment, automation
4. **Multiple Consumption Modes:** API, CLI, IDE, ChatOps, MCP Server, Agent/Chatbot
5. **Approval Workflows:** Human-in-the-loop for sensitive operations
6. **Runbook Automation:** Execute pre-approved remediation actions

Non-Functional Requirements

1. **Security:** Read-only permissions (except bot namespace), IAM least-privilege
2. **Data Residency:** No data egress to public AI endpoints
3. **Local LLM Hosting:** Ollama support for air-gapped environments
4. **Audit Trail:** Complete logging of all actions
5. **Multi-Cloud:** AWS, Azure, GCP support
6. **Enterprise-Ready:** Production-grade error handling, monitoring, testing

Architecture Components

1. Core Agent Framework

- **Agent Engine:** Orchestrates LLM calls, tool execution, planning
- **Tool System:** Extensible tool registry with Kubernetes, cloud, observability tools
- **Memory/Context:** Conversation history, state management

- **Planning:** Multi-step task decomposition and execution

2. LLM Integration Layer

- **Unified Interface:** Abstract LLM provider differences
- **Provider Support:**
 - OpenAI (GPT-4, GPT-3.5)
 - Anthropic (Claude 3.5 Sonnet, Claude 3 Opus)
 - Ollama (Local models: Llama 3, Mistral, etc.)
- **Fallback/Routing:** Smart routing based on task complexity, availability
- **Token Management:** Budget tracking, rate limiting

3. SRE Tool Suite

- **Kubernetes Tools:** kubectl operations, pod logs, events, resource status
- **Cloud Provider Tools:** AWS (CloudWatch, EKS), Azure (AKS), GCP (GKE)
- **Observability:** Prometheus queries, log analysis, trace inspection
- **Network Diagnostics:** VPC flow logs, DNS checks, connectivity tests
- **Migration Assessment:** Workload inventory, complexity scoring

4. Approval & Guardrails

- **Approval Engine:** Request approval for write operations
- **RBAC Integration:** Kubernetes RBAC, IAM roles
- **Blast Radius Controls:** Scope limits, dry-run mode
- **Audit Logging:** Complete action trail

5. API & Interfaces

- **REST API:** FastAPI-based server
- **CLI:** Command-line interface
- **MCP Server:** Model Context Protocol integration
- **ChatOps:** Slack/Teams integration hooks
- **IDE Extensions:** VSCode/IntelliJ plugin architecture

Technology Stack

Backend

- **Language:** Python 3.11+
- **Framework:** FastAPI (API), LangChain/LangGraph (Agent)
- **LLM SDKs:** OpenAI SDK, Anthropic SDK, Ollama client
- **Kubernetes:** kubernetes-python client
- **Cloud SDKs:** boto3 (AWS), azure-sdk, google-cloud-sdk

Infrastructure

- **Containerization:** Docker
- **Orchestration:** Kubernetes (EKS/AKS/GKE)
- **Secrets:** AWS Secrets Manager, HashiCorp Vault
- **Observability:** Prometheus, Grafana, CloudWatch

Security

- **Authentication:** OIDC, IAM roles
- **Authorization:** RBAC, policy enforcement
- **Encryption:** TLS in-transit, KMS at-rest

Implementation Phases

Phase 1: Core Infrastructure (MVP)

- Project structure and dependencies
- LLM integration layer (OpenAI, Anthropic, Ollama)
- Basic agent framework with tool execution
- Simple CLI interface

Phase 2: SRE Tools

- Kubernetes operations tools
- Observability tools (Prometheus, logs)
- Network diagnostics tools
- Cloud provider integrations

Phase 3: Enterprise Features

- Approval workflows
- Audit logging
- RBAC integration
- Error handling and resilience

Phase 4: Consumption Modes

- REST API server
- Enhanced CLI
- MCP Server implementation
- ChatOps integration

Phase 5: Advanced Features

- Migration assessment tools
- Runbook automation
- Multi-step planning
- IDE extensions

Security Architecture

Permissions Model

- **Read-Only:** All cluster resources (except bot namespace)
- **Write:** Only in dedicated bot namespace
- **Cloud Resources:** Read-only IAM roles
- **Secrets:** Scoped access via service accounts

Data Residency

- **No External Egress:** Block public AI endpoints when using local LLMs
- **Private Subnets:** LLM hosting in isolated network
- **Encryption:** TLS for all communications, KMS for data at rest

Audit Trail

- **Action Logging:** All tool executions logged
- **Decision Logging:** Agent reasoning and planning logged
- **Compliance:** Retention policies, SIEM integration

Key Design Decisions

1. **Agent Framework:** LangGraph for complex multi-step workflows
2. **Tool Architecture:** Function-calling compatible tools with JSON schemas
3. **LLM Abstraction:** Unified interface supporting multiple providers
4. **Approval Pattern:** Async approval queue with timeout/escalation
5. **MCP Integration:** Standard protocol for IDE/tool integration
6. **Deployment:** Kubernetes-native with Helm charts

Success Metrics

1. **Functionality:** All MVP features implemented and tested
2. **Multi-LLM:** Seamless switching between OpenAI, Anthropic, Ollama
3. **Security:** Passes security review, no permission violations
4. **Performance:** <5s response time for simple queries
5. **Reliability:** 99.9% uptime for API endpoints
6. **Documentation:** Complete API docs, runbooks, examples

Next Steps

1. Set up project structure and dependencies
2. Implement LLM integration layer
3. Build core agent framework
4. Develop SRE-specific tools
5. Add API server and deployment configs
6. Create comprehensive documentation and tests